Database Systems

Physical Database Design and Database Tuning

Last Lecture…

- Query Processing

- Any questions?

This lecture…

- Physical Database Design and Tuning Overview

Introduction…

- Physical Database Design and Tuning focuses on a common goal:
  - Performance
- That is, how well your database design perform on an actual *workload*.

Introduction… (contd.)

- A workload contains:
  - The most important queries and how often they arise
  - The most important updates and how often they arise
  - The desired performance for these queries and updates

Introduction… (contd.)

- The distinction between Physical Database Design and Database Tuning is blurred.
  - Physical Database Design happens during database development (with predicted workloads)
  - Database Tuning occurs when in use (with actual workloads).

- However, at both stages a common set of *techniques* are used.

Introduction… (contd.)

- Techniques used in Physical Database Design:
  - Index selection and creation*
  - Tuning the conceptual schema
    - Alternative normalised schema
    - Denormalisation
    - Vertical partitioning
    - Horizontal partitioning and views
  - Query re-write

* We will study only index selection and creation in this course

Index Selection

- Index selection requires
  - An understanding of DBMS capabilities
    - Indexing techniques available
    - E.g. SQL Server 2008 supports only B+ Tree indexes
  - Workings of the query optimizer
  - Consider the impact of indexes on updates

- An index may speed-up or slow-down update operations
  - Speed-up: If used in the selection
  - Slow down: Overhead of maintaining index entries when updating data

Index Selection… (contd.)

- Choices in index selection:
  - What relations to index?
  - What are the search key fields in relations?
  - Consider multi-attribute search keys.
  - What type of index?
    - Clustered or unclustered?
    - Tree or hash?
  - Impact on updates.

Index Selection Guidelines

- G1: Whether to index?
  - Index only relations that benefit queries or updates

Example:

```
SELECT …
FROM Emp, Dept, …
WHERE …
```

Emp, Dept, … may benefit from indexes

Index Selection Guidelines… (contd.)

- G2: Choice of Search Key?
  - Attributes in the WHERE clause are candidates for search-keys!
    - Equality selection suggests a Hash index
    - Range selection suggests a B+ Tree index

Examples:

```
SELECT …
FROM  …
WHERE e.empno = 123…
```

Suggests a hash index

```
SELECT …
FROM  …
WHERE e.salary > 20000 …
```

Suggests a B+ Tree index

Index Selection Guidelines… (contd.)

- G3: Multi-Attribute Search Keys?*
  - If WHERE clause contains selections from more than one attribute of a relation
  - Enables INDEX-ONLY plans (discussed later)

  *Order of attributes in search key is important!

- Example:

```
SELECT …
FROM Emp e, …
WHERE e.age = 70 AND e.salary > 20000
```

May benefit from multi-attribute search key

Index Selection Guidelines… (contd.)

- G4: Whether to cluster?*

  * Only one index per relation can be clustered. Must choose it with care!

  - Range queries are most likely to benefit
  - Clustering doesn't help INDEX-ONLY plans! (discussed later)

Index Selection Guidelines… (contd.)

- Schema: Emp(<u>eno</u>, ename, dno)
  Dept(<u>dno</u>, dname, location)

Query 1:

```
SELECT  e.*
FROM    Emp e
WHERE   e.dno = XXX
```

Query 2:

```
SELECT  d.*
FROM    Dept d
WHERE   d.dno = XXX
```

- Which query benefits from a clustered index?
  - Q1 with Emp<dno> or Q2 with Dept<dno>?
- Clustering is useful whenever many tuples are to be retrieved!

Index Selection Guidelines… (contd.)

- G5: Hash vs. Tree Index?
  - A B+ Tree index supports equality and range queries and is usually supported over Hash indexes unless:
    - Index is used to support in the index nested loops join
    - An important equality query and no range queries involving the search key attributes
- G6: Balancing the cost of index maintenance
  - Consider dropping an index if it slows down frequent update operations

Index Selection Guidelines… (contd.)

- G7: When considering a join condition:
  - Hash index on inner is very good for Index Nested Loops.
    - Should be clustered if join column is not key for inner, and inner tuples need to be retrieved.
  - Clustered B+ tree on join column(s) good for Sort-Merge
    - Avoids sorting step

Index Selection (Examples)

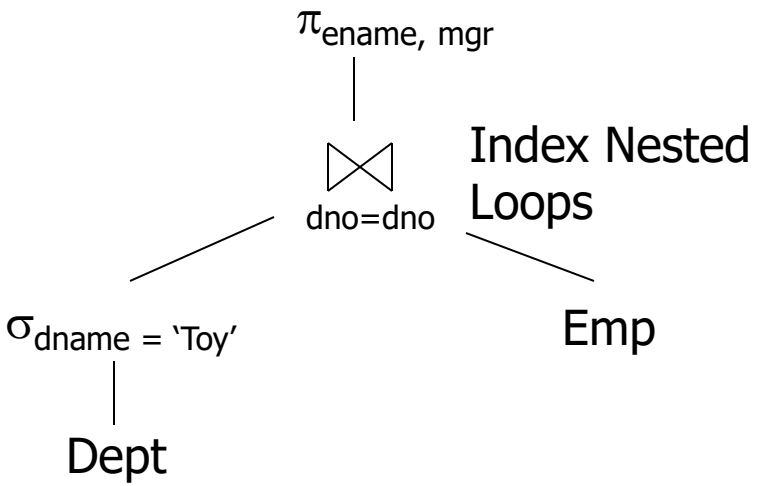
- Example 1:

```
SELECT    e.ename, d.mgr
FROM      Emp e, Dept d
WHERE     d.dname = 'Toy' AND
          e.dno = d.dno
```

- What indexes should we select?

Example 1… (contd.)

- Hash index on Dept<dname> supports efficient retrieval of d.dname = 'Toy'.
- Do we need an index on Dept<dno> or Emp<dno> or both?
- Query Plan:



Example 1… (contd.)

- Index on Dept<dno> is not needed after the d.dname = 'Toy' selection
- Hash index on Emp<dno> is sufficient for the INL join with Emp as the inner relation
- Indexes:
  - Dept<dname>: Clustered?
    - No. Since tuples that satisfy d.dname = 'Toy'is small.
  - Emp<dno>: Clustered?
    - Yes. Many tuples in Emp satisfy the condition e.dno = 'Toy'. Cost of retrieving these tuples can be large without clustering.

Example 1… (contd.)

- Assume that 200 employees work in the "Toy" department.
- Cost of Index Nested Loops Join with an *unclustered index* can be upto

  $\lceil 1.2 + 200 \rceil$ Disk I/Os = 202 Disk I/Os

  where 1.2 I/Os for hash index scan, 1 I/O for tuple retrieval
- Cost of Index Nested Loops Join with a *clustered* index

  $\lceil 1.2 + 2 \rceil$ = 4 Disk I/Os

  where 1.2 I/Os for hash index scan, assuming 100 tuples fit in 1 page (therefore, 200 tuples fit in 2 pages)
- Clustering is useful whenever many tuples are to be retrieved!

Example 2

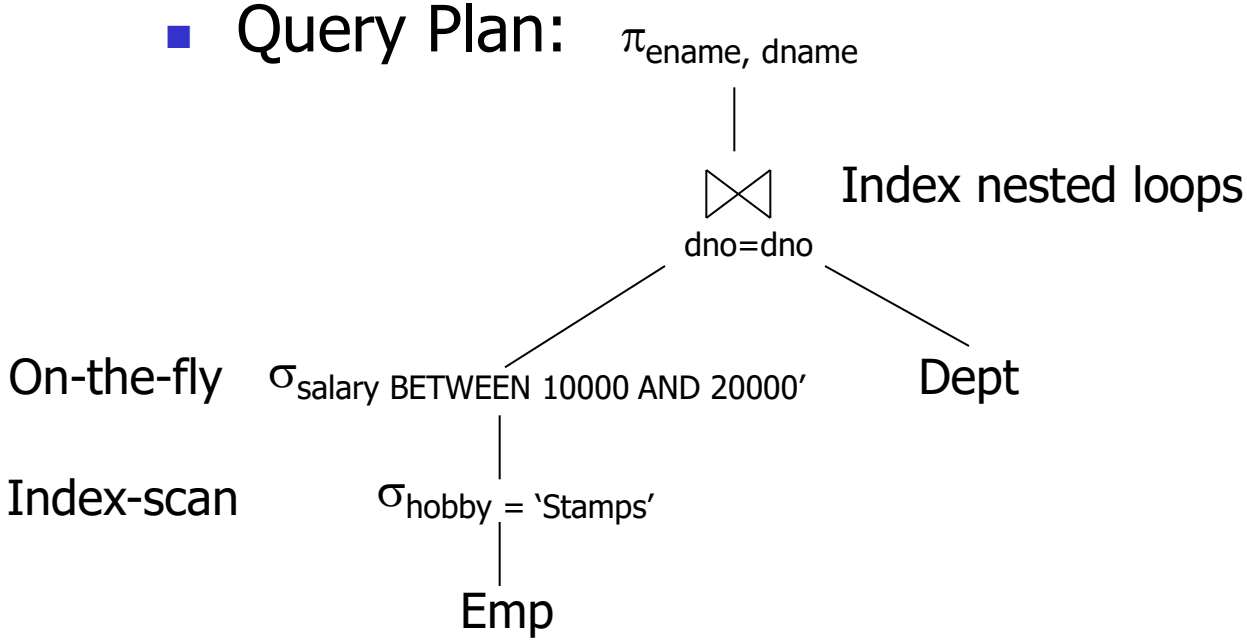
- Example 2:

```
SELECT   e.ename, d.dname
FROM     Emp e, Dept d
WHERE    e.sal BETWEEN 10000 AND 20000 AND
         e.hobby = 'Stamps' AND e.dno = d.dno
```

- What indexes should we create?
  - Similar to example 1, Emp should be outer relation
  - Hash index on Emp<hobby>? or B+ Tree index on Emp<salary>? Depends on *selectivity* of the condition.
  - As a rule of thumb, equality selection is considered to be more selective ➔ create a hash index on Emp<hobby>
  - If both indexes are available, optimizer will choose the more selective index (may be an alternative if selectivity of conditions are unknown)

Example 2… (contd.)

- Query Plan:



- As both examples indicate, our choice of indexes is guided by the plan(s) that we expect an optimizer to consider for a query. *Have to understand optimizers!*

Index-Only Plans

- A number of queries can be answered without ever retrieving tuples from the relations.
- Scanning only the index records are sufficient
- Clustering does not affect scanning index records.
  - Why?
  - An unclustered index is sufficient for index-only scans!
- Usually, index-only scans are very efficient!
  - Why?

Index-Only Plans… (contd.)

- Example 3:

```
SELECT    D.mgr
FROM      Dept d, Emp e
WHERE     d.dno = e.dno
```

- Note that no attributes of Emp are retained.
- If Index Nested Loops Join using Emp as the inner relation with an index on Emp<dno>, then we do not need to retrieve any tuples from Emp relation (index-only plan).
- An unclustered index on Emp<dno> is sufficient!
- Any further improvements?

Index-Only Plans… (contd.)

- Example 4:

```
SELECT    d.mgr, e.eid
FROM      Emp e, Dept d
WHERE     e.dno = d.dno
```

- INLJ with Emp<dno> as inner relation may be expensive without clustering.
- However, a B+ tree index on Emp<dno, eid> enables an index-only scan!

Tools for DBA

- Lots of tools exist for assisting in tuning and index selection

- Example
  - SQL Server's Database Tuning Advisor
  - Etc.